DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Jingyao Zhang
School of Cyber Science and Engineering
Southeast University
Nanjing, China
zhangjingyao@seu.edu.cn

Abstract— Compiler-instrumented defenses, such as stack canaries, Control-Flow Integrity (CFI), and Pointer Authentication (PAC), are vital for mitigating memory-safety exploits. However, these defenses can fail silently—leaving compiled binaries functionally intact but security-compromised, without causing any compiler crashes or execution divergence. For instance, in CVE-2023-4039, dynamic stack allocation silently bypassed the GCC stack protector on AArch64. Such failures expose a fundamental oracle gap: conventional crash and differential testing only check functional correctness, making them completely blind when an optimization pass silently weakens or removes a defense contract.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent compiler defense failures. Following a transparent and verifiable pipeline, DEFUZZ organizes testing into four core steps: (1) Invariant Generation: We extract formal security invariants from compiler corpora, architectural specifications, and historical patches using a hybrid CoT and RAG pipeline. (2) Compiler Source Mapping: With these invariants established, we map them directly to relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. (3) Programmatic Oracle Construction: Next, we turn this mapped logic into target-specific static and dynamic checkers across target ISAs (x86-64, AArch64, RISC-V), strictly enforcing zero LLM involvement in bug adjudication to eliminate false positives from model hallucination. (4) Agentic Directed Fuzzing: Finally, our agentic directed fuzzing coordinates specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations under deterministic oracle evaluation.

Evaluated on GCC and LLVM across multiple architectures, DEFUZZ discovered [TBD: N] previously unknown silent compiler defense failures, of which [TBD: M] have been confirmed upstream. These results underscore the necessity of integrating rigorous, invariant-grounded oracles into compiler testing pipelines.

I. INTRODUCTION

Memory-safety bugs remain a pervasive threat to C and C++ software even after rigorous auditing and testing. Compiler-instrumented defenses, such as stack canaries, Control-Flow Integrity (CFI), and Pointer Authentication (PAC), provide an essential line of defense when such vulnerabilities are triggered. We refer to these mechanisms as compiler defenses and study whether their emitted machine code preserves the intended security guarantees across middle-end transformations and backend lowering.

Depending on the mechanism, instrumentation is introduced in middle-end representations such as GIMPLE or LLVM IR, or during target-specific lowering. The compiler inserts runtime checks, constrains memory layouts, or emits metadata to enforce a particular security policy. For example, a stack canary is verified before a function returns to detect contiguous buffer overwrites that cross the guard.

Defense implementations are themselves complex software components and can harbor defects. Their effectiveness depends on both the instrumentation logic and its compatibility with the target instruction set architecture (ISA) and application binary interface (ABI). Flawed checks, misplaced guards, or incompatible assumptions across compilation stages can silently weaken a defense. The central question is whether the emitted implementation preserves its security guarantees under the concrete semantics and conventions of the target architecture.

A. Silent Defense Failures

A compiler defense fails *silently* when compilation and ordinary functional tests succeed despite a violation of its security guarantees. CVE-2023-4039 exemplifies this problem: an overflow in a variable-length array (VLA) or `alloca` buffer could overwrite a saved return address without triggering the GCC stack protector on AArch64 [1]. The program still behaves correctly on non-attacking inputs, concealing the missing protection. We examine this faulty layout in detail in §II.

Csmith [2] and YARPGen [3] use differential testing to expose compiler errors that change program behavior. Such output comparisons do not establish whether a defense remains effective: correct functional outputs can coexist with broken protection. This is the *oracle gap* for compiler defense testing.

B. Three Coupled Challenges

Our starting point is a *security invariant*: a property that a defense must satisfy under its threat model. Testing such properties can expose missing protection without requiring a functional miscompilation. Turning this idea into a systematic testing method presents three coupled challenges.

Deriving Security Invariants. Defense requirements are scattered across compiler documentation, source comments and assertions, ISA and ABI specifications, and historical fixes.

They are rarely available as executable properties. Deriving an invariant requires identifying the protected asset, activation conditions, and required guarantee, while distinguishing security intent from implementation details that may themselves be faulty.

Constructing Invariant-Directed Tests. An invariant needs both a checker that recognizes a violation and a seed that exercises the relevant conditions. Generic program generation does not explicitly target this combination. For the stack protector, a seed must preserve the allocation and control-flow structure needed to expose a faulty guard placement, and the checker must observe that placement or its effect.

Accounting for ISA Semantics. The same security requirement can depend on different instruction semantics, register conventions, and stack layouts. A separate checker is therefore needed for each applicable ISA, even for the same invariant, to distinguish violations from valid target-specific implementations. The defense mechanism × ISA matrix therefore expands both the search space and the semantic work needed to interpret each result.

C. Our Approach

DeFuzz addresses these challenges with evidence-grounded invariant extraction, invariant-directed agentic testing, and ISA-aware checkers. Offline, it extracts candidate properties from compiler source, specifications, and historical bug reports and patches. Confirmed root causes also serve as retrieval probes for related implementations. Each accepted invariant retains its source evidence and is specialized to a concrete defense; transferring a failure pattern does not make the resulting invariant universal across mechanisms.

Online, each invariant and its checker suite define a testing objective. Large Language Model (LLM) agents use source-search tools and the invariant context to construct seeds, then revise them using measured coverage and checker feedback. This motivates the agentic design: generating a syntactically valid program is insufficient when the test must preserve specific semantic conditions, and one-shot generation cannot adapt to whether those conditions were reached. The agents perform this contextual refinement, while a fixed pipeline controls compilation and checking.

Static and dynamic checkers interpret evidence using target-specific instruction and ABI assumptions. Their metadata determines ISA applicability and routing, so unsupported cases are not mistaken for successful protection. Bug claims require deterministic binary or runtime evidence, not an LLM verdict. A versioned blackboard records seeds, feedback, and results for replay and ablation.

D. Contributions

In summary, this paper makes the following contributions:

- **Evidence-Grounded Security Invariants.** We derive defense-specific properties from documented requirements and historical failures, using root-cause retrieval to identify related candidates with explicit provenance (§III-A, §III-A3).

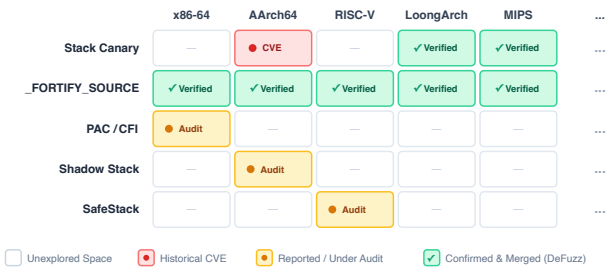


Fig. 1. The defense mechanism × ISA testing space: correctness on one architecture does not establish correctness on another, creating a vast matrix of unexplored compiler defenses.

- **Invariant-Directed Agentic Testing.** We couple executable checkers with semantic seed generation and feedback, using explicit orchestration to make the search traceable and its components ablatable (§III-E).
- **ISA-Aware Oracles.** We operationalize security invariants as separate static or dynamic checkers for each applicable ISA, with deterministic evidence, addressing the oracle gap across supported targets (§III-B).
- **Empirical Discovery.** On GCC and LLVM across multiple architectures, DeFuzz discovered **[TBD: N]** previously unknown silent compiler defense failures, of which **[TBD: M]** were confirmed upstream.

II. Background and Motivation

A. Compiler-Instrumented Defenses

A compiler-instrumented defense introduces checks, layout constraints, or metadata at compile time to enforce a security policy at run time. Enforcement may also require runtime libraries, operating-system support, or hardware features. Stack canaries use guard comparisons, CFI restricts indirect control-flow targets, and PAC uses signing and authentication instructions. The presence of such instrumentation alone does not establish that the intended protection holds.

Defense logic can span target-independent IR and backend representations such as RTL, SelectionDAG, or Machine IR. Lowering must respect instruction semantics and ABI conventions for registers, stack alignment, and call frames. For instance, dynamic allocations through `alloca` or VLAs require target-specific stack-pointer adjustments and can change the relation between buffers, saved registers, and guards. A property preserved by one backend may therefore fail in another. Figure 1 illustrates this mechanism × ISA testing space: defense implementations may share target-independent middle-end logic, but their lowered machine-code behavior demands separate, target-specific validation across distinct backend architectures.

B. Sources of Security Invariants

Security invariants express required protection, rather than merely describing behavior observed in an existing binary. Their evidence comes from three sources: compiler source and documentation, ISA and ABI specifications, and historical

bug reports with their fixes. Documentation and specifications describe intended constraints; source comments and assertions identify where they apply; bug reports and patches expose assumptions that an implementation failed to preserve. Because the implementation under test can be wrong, its current behavior is not by itself a specification.

Recent work also separates property generation from verification. PropertyGPT [4] retrieves human-written smart-contract properties to generate new candidates and checks them with a dedicated prover. CIll [5] uses model-checking feedback to refine LLM-generated inductive invariants. These approaches motivate pairing generated properties with independent checking. Our task differs from proving a program assertion: we extract the security requirements of compiler defenses and search for compiled artifacts that violate them. An invariant must state its mechanism, applicability conditions, required property, and supporting evidence before it can guide a checker and a test seed.

C. Threat Model and Scope

We evaluate emitted defense code under the assumptions of each defense mechanism:

- **Assumptions:** The application contains a memory-safety vulnerability that an attacker can trigger. The defense is enabled and its required runtime and hardware support is present. Attacker capabilities are limited to the threat model of the mechanism being tested; a stack canary, for example, does not promise to stop arbitrary writes that skip the guard.
- **In-Scope:** We study silent failures in compiler-instrumented defenses: compilation and ordinary functional tests succeed, but the emitted code violates a security invariant under the documented preconditions. Tests may inspect binary structure or exercise a violation at run time. They establish the failed protection without requiring an end-to-end exploit.
- **Out-of-Scope:** We do not search for application-level vulnerabilities or intentional source-level bypasses such as inline assembly that disables protection. Hardware vulnerabilities, side-channel attacks, and business logic flaws are also excluded. We evaluate standard defense emissions and exclude interactions between compilation optimization options and defense mechanisms.

The source corpus used for invariant extraction is pinned to GCC 16.1; bug-testing results are tied to the compiler version and target configuration recorded for each run.

D. Silent Failure: A Representative Case

CVE-2023-4039 illustrates the connection between a defense requirement, its architectural assumptions, and a testable violation [6]. For a contiguous stack overflow covered by the stack-protector policy, the guard must lie on the overwrite path to the saved return address. In the affected AArch64 layout, the saved return address lies at a higher address than the VLA or `alloca` buffer but below the canary (Figure 2). An overflow toward higher addresses can therefore corrupt

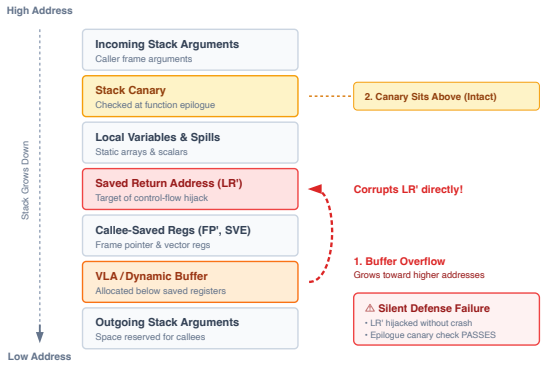


Fig. 2. CVE-2023-4039: an overflow from the VLA reaches the saved return address before the canary, leaving the guard unchanged.

the return address without changing the guard. The canary comparison still executes and passes.

The report traces this failure to a mismatch between target-independent guard placement and the AArch64 frame layout. It suggests both a seed containing a dynamic allocation and a checker that resolves the relative locations of the allocation, guard, and saved return address. Merely finding a canary comparison in the binary would miss the failure.

A separate failure, GCC PR-96191 [7], concerns a canary value left in a temporary register after the AArch64 stack-protector check. The report cites the machine-description requirement that intermediate values must not survive the protection sequence. This yields a different invariant, about register state rather than stack layout, and requires a checker that follows the relevant register updates. The two cases show why the security property and its ISA-specific observation must both be explicit.

E. Why Existing Methods Miss It

Crash and differential testing can reveal functional miscompilations, but matching outputs on non-attacking inputs do not validate a defense. Moreover, attack-triggering executions may involve source-level undefined behavior, making ordinary cross-compiler output agreement unsuitable as the security oracle. Tests must instead check the protection promised by the enabled defense.

Static checking can detect a known violation when its rule models the relevant semantics. A symbol or instruction-presence rule, however, cannot establish correct guard placement or register cleanup. Extending such checks requires defense-specific properties and target-specific interpretation.

Generic seed generation does not directly connect those properties to the program structures that exercise them. Random generation may reach such structures, but coverage alone does not say whether a seed tests the assigned invariant. This motivates agents that inspect source context and revise seeds from checker feedback. Their role is to direct the search; executable checkers remain responsible for deciding whether the generated artifact violates the defense requirement.

III. Design of DeFuzz

In this section, we present the design of DeFuzz, an invariant-directed framework for discovering silent failures in compiler-instrumented defenses. Figure 3 illustrates its overall workflow, which consists of four core steps: invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract formal security invariants from compiler corpora and historical defects using a hybrid CoT and RAG pipeline. With these invariants established, we map them to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Next, we turn this mapped logic into programmatic bug oracles. We implement these oracles as static and dynamic checkers across target ISAs to determine whether program seeds expose defense failures, mitigating false positives from model hallucination. Finally, our agentic directed fuzzing coordinates multiple specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations, with our programmatic oracles evaluating and verifying each candidate. We discuss each step in detail in the following subsections.

A. Phase 1: Security Invariant Extraction

The offline preparation phase begins by systematically transforming unstructured compiler corpora into a catalog of verifiable security invariants.

1) Formal Invariant Contract: A *security invariant* is a property that a compiler-instrumented defense must satisfy under its designated threat model to preserve its security guarantees. Violating a security invariant indicates that the defense has been silently weakened or bypassed in the emitted machine code or at run time, regardless of whether compilation completes successfully or functional test suites pass.

To ensure deterministic evaluation and downstream toolchain integration, every invariant ϕ is formalized under a uniform schema:

$$\phi = \langle \mathcal{M}, \mathcal{A}, \mathcal{P}_{\text{cond}}, \mathcal{S}_{\text{inv}}, \mathcal{O}_{\text{phenom}}, \mathcal{E}_{\text{prov}} \rangle \tag{1}$$

where $\mathcal{M}$ denotes the target defense mechanism (e.g., Stack Canary, SafeStack, PAC/CFI), $\mathcal{A}$ defines the applicable ISAs, $\mathcal{P}_{\text{cond}}$ specifies the activation preconditions in the source code (e.g., dynamic buffer allocations via VLA or `alloca`), $\mathcal{S}_{\text{inv}}$ represents the verifiable invariant statement, $\mathcal{O}_{\text{phenom}}$ specifies the externally observable phenomenon of a violation, and $\mathcal{E}_{\text{prov}}$ retains the supporting provenance evidence.

Crucially, DeFuzz strictly separates the observable phenomenon $\mathcal{O}_{\text{phenom}}$ from the concrete detection recipe. For example, "the canary guard does not precede the saved frame pointer on the stack overwrite trajectory" is an observable phenomenon, whereas "inspect the stack layout offsets in the DWARF debug frame or binary disassembly" is a target-specific detection recipe implemented by the oracle suite (§III-B).

2) Candidate Extraction via Segmented CoT: The invariant corpus is synthesized from three primary knowledge domains: (1) compiler source code, including developer comments, internal AST/IR lowering logic, and runtime assertions; (2) official architectural and ABI specifications; and (3) historical vulnerability reports (CVEs) and their corresponding compiler patches.

Due to the scale of production compilers, semantic requirements for a single defense are often fragmented across middle-end transformations (e.g., GIMPLE or LLVM IR passes) and target-specific backend lowering files (e.g., machine descriptions and frame layout calculations). To extract candidates systematically, DeFuzz partitions compiler source trees along mechanism and function boundaries. A segmented Chain-of-Thought (CoT) prompting strategy [8] then guides the language model to examine each partition independently, isolating the protected asset, required preconditions, and security guarantee. This phase produces *candidate invariants*, each tagged with its source file and line-level evidence to eliminate hallucination before downstream validation.

3) Cross-Mechanism Transfer via Hybrid RAG: Segmented CoT review provides broad coverage within known mechanism boundaries, but it cannot readily cluster isolated code fragments that share an identical underlying failure mode across different defenses or compiler backends. To bridge this divide, DeFuzz incorporates a complementary Retrieval-Augmented Generation (RAG) [9] pipeline centered on *abstract-failure-mode transfer*.

Rather than performing entity-centric retrieval based on variable or API names (which confines search results to the original defense), DeFuzz distills confirmed historical bugs into abstract semantic signatures. The transfer pipeline executes across four formal stages:

1) **Distillation:** A confirmed root cause is abstracted into a mechanism-agnostic failure signature. For example, CVE-2023-4039 is distilled from "AArch64 VLA overwrites return address before canary" into the generalized property: "dynamically sized local allocations bypass statically computed security boundaries."
2) **Analogy Filter:** A semantic filter scans retrieved candidates across other defenses (such as SafeStack, ShadowCallStack, or PAC), discarding superficial lexical matches and verifying whether the structural failure pattern applies.
3) **Specialization:** The abstract property is instantiated within the concrete target implementation, mapping abstract boundaries to concrete data structures (e.g., shadow-stack pointer adjustments or PAC signature verification sequences).
4) **Entailment Verification:** A formal check confirms that the specialized invariant statement logically follows from the cited source evidence.

a) Dual-Aspect Query Formulation: To query the codebase effectively, each distilled failure mode is encoded into a dual-aspect representation: a dense semantic summary (`root_cause_phrase`) and sparse mechanism-specific to-

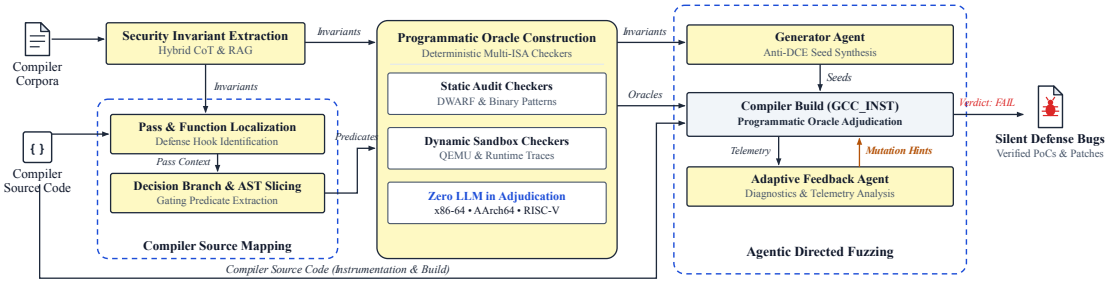


Fig. 3. Overview of DEFUZZ. The framework operates across four core steps: (1) extracting security invariants from compiler corpora and historical defects via a hybrid CoT and RAG pipeline; (2) mapping invariant logic to compiler passes and decision branches; (3) constructing programmatic static and dynamic oracles across target ISAs; and (4) performing agentic directed fuzzing, where a generator agent synthesizes probe seeds and a feedback agent steers iterative exploration under programmatic oracle verification.

TABLE I
TRANSFERRED INVARIANTS BY RETRIEVAL BACKEND (24 PROBES, 4,496 GCC-16.1 CHUNKS).

Category	Count	Note
Intersection (BM25 $\cap$ embed)	5	Same seed & chunk, isomorphic statement
BM25-only	4	Embedding missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	**11**	$9+7-5$; all verified as novel

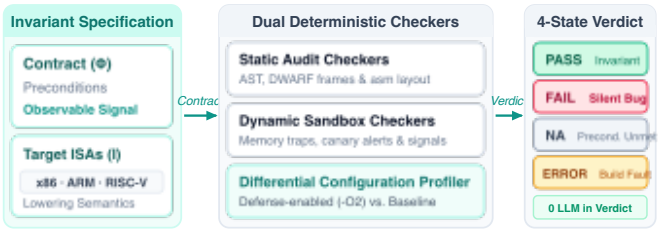


Fig. 4. Multi-ISA deterministic oracle architecture. Invariant contracts are specialized across target ISAs and instantiated into dual static audit and dynamic sandbox checkers, yielding a deterministic four-state verdict without LLM adjudication.

kens (`critical_tokens`, such as register names, frame macros, and pass flags).

We evaluate sparse BM25 and dense embedding retrieval over 24 probe seeds across 4,496 GCC-16.1 source chunks. BM25 accurately retrieves explicit token matches (e.g., `ix86_zero_call_used_regs`, `SUBREG_PROMOTED`), whereas dense retrieval uncovers semantically isomorphic implementations whose lexical tokens diverge (e.g., ARM CMSE clear-set constructors and RISC-V saturating truncations). Deduplicating by `(seed_id, chunk_id)` and invariant statement, the two paths yield 5 in the intersection, 4 BM25-only, and 2 embedding-only, producing 11 deduplicated novel invariants in their union (Table I).

4) Grounding Gates and Catalog Maintenance: Candidates from both segmented CoT review and cross-mechanism RAG must pass three static grounding gates before entering the catalog Φ:

- **Gate 1 (Source Evidence Verification):** Verifies that cited source files, AST structures, or specification sections explicitly corroborate the stated precondition and guarantee.
- **Gate 2 (Observability Verification):** Ensures that the violation phenomenon $\mathcal{O}_{\text{phenom}}$ is deterministically observable in the compiled binary or runtime trace, without requiring speculative execution assumptions.
- **Gate 3 (Novelty Deduplication):** Measures cosine similarity against existing invariant statements, rejecting candidates with similarity scores exceeding 85.0% to prevent redundant test targets.

Accepted invariants are stored in the persistent catalog $\Phi = \{\phi_1, \phi_2, \ldots, \phi_n\}$. Furthermore, whenever the online fuzzing campaign confirms a new zero-day vulnerability, its verified root cause is distilled and serialized back into the probe pool, enabling continuous reinforcement of the invariant catalog across successive testing cycles.

B. Phase 2: Multi-ISA Oracle Formulation

Accepted invariants are operationalized into deterministic contracts. For each invariant and supported ISA, DeFuzz generates specialized static and dynamic checkers (Figure 4).

1) Oracle Contract and Four-State Verdict Matrix: To evaluate compiler-instrumented defenses without human intervention or LLM adjudication bias, DeFuzz formalizes the oracle as a deterministic contract. Let $\mathcal{P}$ denote the space of compiled program binaries, $\mathcal{I} = \{\text{x86-64}, \text{AArch64}, \text{RISC-V}\}$ be the set of target ISAs, and $\mathcal{S} \subseteq \Phi$ be the set of accepted security invariants. The oracle is defined as a deterministic function:

$$\mathcal{O} : \mathcal{P} \times \mathcal{I} \times \mathcal{S} \longrightarrow \{\text{PASS}, \text{FAIL}, \text{NA}, \text{ERROR}\} \quad (2)$$

Each verdict carries strict semantic guarantees:

- **PASS:** The compiled binary strictly satisfies the security invariant under the assigned architectural assumptions.

- **FAIL:** A deterministic violation of the security invariant is proven, confirming a silent compiler defense bug.
- **NA (Not Applicable):** The target defense mechanism was inactive or the seed did not trigger the precondition required to evaluate the invariant.
- **ERROR:** Compilation aborted due to syntax errors or environment faults; this verdict is recorded as an infrastructure diagnostic and never promoted to a bug claim.

Crucially, LLM agents are strictly barred from adjudicating or overriding verdicts. All bug claims require programmatic verification by the oracle function $\mathcal{O}$, eliminating false positives stemming from model hallucination.

2) Dual Static and Dynamic Checkers: DeFuzz constructs dual checker implementations tailored to the observability requirements of each invariant:

- **Static Checkers:** Inspect unexecuted compiler outputs, including AST dumps, GIMPLE representations, LLVM IR, assembly listings, stack frame layouts, and ELF symbol tables. For stack canaries, the static checker audits whether the canary allocation slot precedes all local buffers on the stack overwrite path and checks the presence of guard symbols (e.g., `__stack_chk_fail`).
- **Dynamic Checkers:** Execute compiled probe binaries within isolated sandboxes, instrumented with runtime overflow triggers, canary corruption traps, or memory-fault signal handlers. For CFI and PAC, dynamic checkers invoke indirect calls with forged signatures or invalid targets, verifying that the hardware or runtime trap is raised deterministically.

Invariants whose violation signals can be observed both statically and dynamically are implemented with redundant checkers to cross-validate results.

3) Target-Specific Specialization Across ISAs: Although a security invariant expresses a universal defense guarantee, its machine-level enforcement depends fundamentally on target-specific instruction semantics, register conventions, and stack layout rules. For each invariant $s \in \mathcal{S}$ and each applicable ISA $i \in \mathcal{I}$, DeFuzz synthesizes a specialized checker implementation $C_{s,i}$.

On x86-64, return addresses are pushed directly onto the stack by the `call` instruction, placing the return slot immediately above the saved frame pointer. In contrast, on AArch64 and RISC-V, the return address is held in a dedicated link register (`x30` or `ra`) and saved onto the stack only during non-leaf function prologues. Consequently, an AArch64 stack protector checker must track whether dynamic frame adjustments (e.g., for VLAs) place buffers above the saved link register slot, whereas an x86-64 checker focuses on displacement relative to the base pointer. By generating specialized checkers tagged with ISA metadata, DeFuzz prevents valid architectural variations from being misclassified as defense failures.

4) Differential Configuration Profiling: The oracle suite incorporates a mechanism-level aggregator that combines verdicts across checkers and compilation flag profiles. Checkers execute under both defense-enabled flags (e.g., `-fstack-protector-strong`, `-fcf-protection`) and baseline configurations (defense disabled). This differential profiling ensures that detected layout properties or symbol changes are attributable specifically to the defense instrumentation rather than general backend optimization passes. Any verified FAIL across the checker suite triggers a mechanism-level alert carrying full binary and trace evidence.

C. Phase 3: Hierarchical Compiler Source Mapping

1) MCP-Guided Codebase Navigation: Production compilers such as GCC and LLVM comprise millions of lines of complex C/C++ code. Localizing the exact routines where security defense decisions are made presents a severe scalability challenge for automated testing. To navigate this code surface, DeFuzz deploys an autonomous coding agent equipped with a Model Context Protocol (MCP) toolchain.

The MCP toolchain exposes four specialized analysis capabilities: (1) `call-graph` to trace invocation paths across optimization passes; (2) `ast-parser` to inspect compiler abstract syntax trees; (3) `alia ast` for intermediate representation audits; and (4) `ripgrep` for rapid symbol and macro indexing. The agent uses these tools in a read-only environment, formulating hypotheses regarding where defense logic is lowered and validating them against compiler source definitions.

2) Three-Level Decision Hierarchy: Using CallGraph MCP and code retrieval tools, the agent resolves each invariant ϕ into a 3-level hierarchical mapping within the compiler source tree:

- **Level 1: Compiler Pass:** Identifies the high-level middle-end or backend pass governing the defense mechanism (e.g., `pass_stack_protect` in GCC, or `StackProtectorPass` in LLVM).
- **Level 2: Target Functions:** Isolates the specific functions responsible for frame allocation, guard placement, or instruction lowering (e.g., `expand_used_vars()` in GCC, or `LayoutStackNodes()` in LLVM).
- **Level 3: Critical Structs & Conditions:** Pinpoints the exact data structures and conditional predicates governing defense activation and layout decisions, such as `has_vla`, `stack_protect_decl`, and `canary_loc`.

This 3-level hierarchy establishes an exact correspondence between abstract invariant preconditions and concrete compiler decision branches.

D. Phase 4: Compiler Probing and Driver Rebuild

1) Deterministic UID Log Probing: Once the Level-3 critical conditions are mapped, the coding agent automatically instruments the compiler source by inserting deterministic logging probes. Each probe is instantiated using a standardized compile-time macro:

```
DEFUZZ_LOG("SP#a3f2:hit", slot);
```

where `"SP#a3f2"` represents a globally unique identifier (UID) encoding the defense mechanism, file, function, and

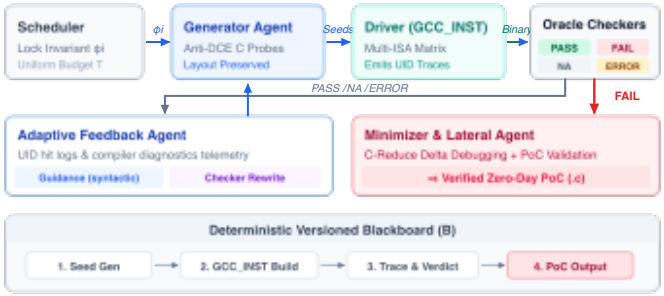


Fig. 5. Online closed-loop agentic fuzzing workflow. A deterministic orchestrator schedules invariant targets under fixed budgets. A generator agent synthesizes anti-DCE seeds, which are built by `GCC_INST` and evaluated by the oracle. Failures trigger C-Reduce minimization and lateral amplification, while non-violations feed telemetry into guidance and checker evolution loops.

condition index. These probes execute during compilation passes, recording whether critical branches were taken and capturing relevant internal variables (e.g., stack slot offsets) without altering the compiler's code generation logic or introducing runtime overhead into the emitted target binary.

2) Instrumented Compiler Driver Rebuild (`GCC_INST`): Following probe insertion, the compiler is fully rebuilt from source into an independently executable instrumented compiler driver, denoted `GCC_INST` (or `CLANG_INST` for LLVM). When `GCC_INST` compiles a test seed, it writes a deterministic UID trace stream to disk, providing the fuzzing loop with precise visibility into whether internal defense predicates were exercised.

E. Phase 5: Invariant-Directed Agentic Fuzzing

With the offline invariant catalog, deterministic oracles, and instrumented compiler driver established, DeFuzz executes its online testing campaign. The online phase operates as a closed-loop agentic exploration engine that schedules invariants under uniform time slices, generates semantically targeted seeds engineered to survive compiler optimizations, and continuously adapts based on execution telemetry (Figure 5).

1) Deterministic Orchestration Pipeline: The online fuzzing process is governed by a deterministic orchestrator rather than an autonomous agent. At campaign startup, the orchestrator initializes a target queue Q from the invariant catalog Φ. The orchestrator locks one target invariant ϕ_i at a time alongside its dedicated ISA-specific checker suite, allocating a fixed, uniform time and execution budget T (ensuring equal exploration opportunity across all invariants).

All interactions follow a strict topological order: seed generation $\rightarrow$ instrumented build with `GCC_INST` $\rightarrow$ UID trace collection $\rightarrow$ dual oracle evaluation $\rightarrow$ feedback dispatch. To guarantee reproducibility and auditability, all inter-agent communications and intermediate states flow through a versioned blackboard $\mathcal{B}$. Pinning a blackboard version deterministically reproduces agent inputs, test seeds, compiler traces, and checker results. Algorithm 1 details the complete control flow.

Algorithm 1 Online Closed-Loop Agentic Fuzzing Pipeline

Require: Invariant queue Q, Budget T, Instrumented driver `GCC_INST`, Oracle suite $\mathcal{O}$

Initialize versioned blackboard $\mathcal{B}$, Historical seed pool $\mathcal{H} \leftarrow \emptyset$
for target invariant $\phi_i \in Q$ **do**
  Set round budget $t \leftarrow T$
  while $t > 0$ **do**
    $\mathcal{B}$.seed $\leftarrow$ GeneratorAgent(ϕ_i, $\mathcal{B}$.feedback)
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{\mathcal{B}.\text{seed}\}$
    $\mathcal{B}$.binary, trace $\leftarrow$ Compile(`GCC_INST`, $\mathcal{B}$.seed, ϕ_i.flags)
    $v \leftarrow \mathcal{O}(\mathcal{B}.\text{binary}, \phi_i.\text{ISA}, \phi_i)$
    if $v ==$ FAIL **then**
      $\mathcal{B}$.poc $\leftarrow$ MinimizerAgent($\mathcal{B}$.seed, $\mathcal{O}$, ϕ_i)
      yield ConfirmedBugFound($\mathcal{B}$.poc, ϕ_i)
      LateralDivergence(ϕ_i, $\mathcal{B}$.seed)
      break
    else if $v \in \{\text{PASS}, \text{NA}, \text{ERROR}\}$ **then**
      $\mathcal{B}$.feedback $\leftarrow$ FeedbackAgent(v, trace, $\mathcal{B}$.seed)
      if $\mathcal{B}$.feedback.rewrite_votes $\geq \tau$ **then**
        ϕ_i.checker $\leftarrow$ TriggerCheckerRewrite(ϕ_i.checker)
        ReplaySeedPool($\mathcal{H}$, ϕ_i.checker, `GCC_INST`)
      end if
    end if
    $t \leftarrow t - 1$
  end while
end for

2) Checker-Aware Seed Synthesis and DCE Evasion: Generic random program generators frequently produce test inputs whose security-sensitive constructs are optimized away by compiler transformations. In particular, Dead Code Elimination (DCE) routinely removes unused buffer allocations or ineffectual pointer writes before backend defense passes execute.

To overcome this, DeFuzz employs an LLM-based Generator Agent that synthesizes *checker-aware* C/C++ probe seeds (e.g., `seed_sp_probe.c`). The agent is conditioned on the formal invariant statement ϕ_i, the target ISA, and the Level-3 compiler predicates identified through hierarchical source mapping (§III-C). It systematically incorporates anti-DCE patterns into generated seeds:

- Dynamically sized arrays (VLAs) with run-time dimension bounds passed via command-line arguments (`argc`).
- Escaping buffer pointers through volatile qualifiers and `asm volatile` barriers.
- External function invocations and non-inlinable calls that force the compiler to preserve complete stack frame structures and canary prologue/epilogue checks.

3) Adaptive Telemetry Feedback and Checker Evolution: When the oracle suite returns a non-violation verdict (PASS, NA, or ERROR), execution transitions to the Feedback Agent. The Feedback Agent executes `GCC_INST` via Bash and MCP

tools, extracting multi-dimensional feedback: (1) compiler diagnostics and warnings; (2) probe hit logs recording critical condition trigger counts (hit_count(UID)); and (3) detailed oracle evaluation traces.

The feedback agent drives two closed-loop adaptation mechanisms:

- **Guidance Loop:** If Level-3 condition hits indicate that the seed failed to reach the required compiler path (e.g., `has_vla` evaluated to false), the Feedback Agent generates precise syntactic mutation hints for the Generator Agent, specifying required control-flow or variable-type adjustments for the next round.
- **Checker Rewrite Loop (Evolution):** If the Feedback Agent identifies persistent semantic ambiguity or rule divergence across multiple rounds (accumulating votes $\geq \tau$), it triggers an automated rewrite of the ISA checker. Crucially, upon checker update, the system replays the historical seed pool $\mathcal{H}$ against the newly revised checker to detect previously masked violations.

4) Automated Bug Minimization and Lateral Amplification: When the oracle outputs FAIL, a silent compiler defense violation is confirmed based exclusively on programmatic evidence. The failing seed and its execution trace are transferred to an automated minimization pipeline.

The Minimizer Agent pairs delta-debugging (C-Reduce) with semantic validation, ensuring that reduction passes do not eliminate the critical memory layout or allocation pattern that exposes the bug. Each reduced candidate is recompiled with `GCC_INST` and rechecked by $\mathcal{O}$ before being accepted. The resulting zero-day Proof-of-Concept (PoC) is recorded in the CVE corpus.

a) Lateral Divergence (Bug Amplification): Confirming a bug does not terminate testing for that invariant. Instead, the orchestrator initiates a *Lateral Divergence Loop*, prompting the Generator Agent to synthesize diverse syntactic and structural variants (e.g., mutating boundary types, introducing composite structures, and testing atypical stack alignments) around the identified invariant ϕ_i. This bug amplification mechanism systematically explores whether the underlying defect propagates across sibling optimization passes or alternate backend lowering targets.

IV. Implementation

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

References

[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[4] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[5] Y. Su, T. Bu, Q. Yang, Y. Ci, and E. Tian, "CIll: CTI-guided invariant generation via LLMs for model checking," arXiv preprint arXiv:2602.23389, 2026. [Online]. Available: https://arxiv.org/abs/2602.23389

[6] T. Hebb, "CVE-2023-4039: GCC's -fstack-protector fails to guard dynamic stack allocations on ARM64," https://rtx.meta.security/mitigation/2023/09/12/CVE-2023-4039.html, 2023.

[7] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.

[8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.